

Block-Diffusion OD Planning — Experimental Results

Data: Porto taxi trajectories on a shrunk road network (1,390 vertices, 2,003 edges), ~1.93M shortest paths. Normal graph: `v3-0.1_normal` (= `v4-*_normal`, verified identical). Exceptional scenarios: `v4-0.1_except_k` ($k = 0..99$), each removing 200 edges (`v4-0.05_*`: 100 edges).

Base model: BD3-LM-style block diffusion (semi-autoregressive over blocks, discrete diffusion within each block). Transformer denoiser (6 layers, dim 256, 6.8M params), trained 1 epoch ($\approx 48k$ iters, bs 32, lr $5e-4$) on **normal data only**. Two within-block kernels:

- **mask** — absorbing/[MASK] diffusion (SUBS weighted cross-entropy, first-hitting sampling)
- **uniform (grapheos)** — graph CTMC kernel $Q_t = \exp((A-D) \cdot \beta_t)$ with the state space augmented by an `<end>` state (virtual node, total degree 2.0), D3PM ELBO (KL + CE + connectivity)

Both kernels support **conditional** (origin + destination) and **unconditional** generation (condition dropout $p=0.1$ during training drops both the OD embedding and the clean prefix). Generation is open-ended: blocks are appended until the destination or `<end>` token is emitted (no length model). Evaluation: 1,000 OD pairs, `length_mode=open`, CFG weight $w = 1.0$ (best; see §1).

Metrics: hit = path reaches dst unaided; arrival = endpoint equals dst (after the variant's refinement, if any); valid = every consecutive edge exists in the evaluation graph; EM = valid AND length equals the OD's shortest length; PC = $1/(1 + \text{\#feasible shorter lengths})$; DTW/LCS vs the ground-truth path; len = $|\text{len(gen)} - \text{len(real)}|$.

1. Normal case over block sizes (train: normal, test: normal)

Refinement (§1.1): the tables below use the `refine()` function from `eval_shortest.py` (variant `refine_es`), applied to *all* generated paths: cut the path at the first vertex adjacent to the destination and append the destination, then remove loops. It does not guarantee arrival. Every evaluation also recorded `raw` (no refinement) and `refine_nx` (networkx shortest-path patch to dst, guarantees arrival) variants; all are in `sets_res/*.res`.

1.1 Mask kernel (CFG $w = 1.0$, `refine_es`)

blk	hit	arrival	valid	EM	PC	DTW	LCS	len err
1	0.993	0.996	0.950	0.866	0.901	0.035	21.3	0.33
2	0.973	0.995	0.905	0.828	0.862	0.029	21.6	0.31
4	0.942	0.976	0.855	0.796	0.822	0.036	21.7	0.43
8	0.958	0.981	0.772	0.724	0.746	0.036	21.6	0.42
16	0.920	0.975	0.694	0.643	0.666	0.048	20.7	0.57
32	0.893	0.946	0.508	0.474	0.490	0.060	19.4	0.97
64	0.883	0.943	0.505	0.471	0.488	0.059	19.8	0.95

- Monotonic quality improvement toward the AR limit (blk 1); blk 64 $\approx$ full-canvas diffusion (MDLM).
- blk 2 is the best fidelity/efficiency trade-off (best DTW and length error).
- CFG note: $w = 1.0$ (no guidance amplification) beat $w = 2 / 4 / 8$ decisively at every block size (e.g. blk4 valid: 0.855 / 0.693 / 0.107 / 0.025). The destination is already in-context as a prefix token, so CFG extrapolation only pushes logits off-manifold.

1.2 Uniform kernel + `<end>` state (CFG $w = 1.0$, `refine_es`)

blk	hit	arrival	valid	EM	PC	DTW	LCS	len err
1	0.342	0.460	0.986	0.440	0.590	0.534	11.5	6.30
2	0.139	0.275	0.996	0.493	0.635	0.484	11.0	6.14
4	0.156	0.303	0.989	0.755	0.844	0.324	13.4	6.11
8	0.120	0.208	0.896	0.585	0.704	0.483	10.9	8.86
16	0.118	0.212	0.969	0.658	0.787	0.438	11.7	8.55
32	0.161	0.326	0.994	0.557	0.704	0.401	14.5	6.05
64	0.125	0.266	0.996	0.500	0.659	0.469	13.4	6.37

- Near-perfect validity comes from the adjacency-constrained decode, **but the `<end>` state is miscalibrated: generation stops too early at every block size** (low hit/arrival, 6–9 token undershoot). Caveat: EM/PC score each path against its *own* endpoints, so

early-stopped fragments inflate EM — hit/arrival are the honest columns for this kernel.

- Root cause: the virtual `<end>` node is (weakly) adjacent to every vertex, making it too easy to reach in both the reverse CTMC and the decode. Fix candidates: smaller `-bd_eos_deg`, suppressing `<end>` in the decode unless the model's probability strongly favors it.

1.3 EOS-degree calibration sweep (uniform kernel, blk 4, refine_es)

Retraining the blk-4 uniform kernel with different `<end>`-node degrees d (§4.1 gives the math — d scales the prior propensity to propose `<end>` during the reverse process):

eos_deg d	hit	arrival	valid	DTW	len err
0.05	0.328	0.539	0.996	0.213	3.57
0.1	0.175	0.328	0.987	0.294	5.54
0.25	0.076	0.154	0.984	0.459	8.41
0.5	0.152	0.306	0.992	0.328	6.15
1.0	0.175	0.328	0.997	0.263	5.20
2.0 (default)	0.156	0.303	0.989	0.324	6.11

$d = 0.05$ doubles hit (0.156 → 0.328) and arrival (0.303 → 0.539) and cuts the length undershoot from 6.1 to 3.6 tokens at unchanged validity — the recommended setting. The mid-curve is noisy (0.25 appears to be an unlucky run), but the low-degree end is unambiguous; even smaller d is an open follow-up. Note: all §2/§3 uniform-kernel guidance results below use the older $d = 2.0$ checkpoints.

1.4 No `<end>` state at all — destination as the terminator (uniform kernel)

For **conditional OD planning the destination is itself the natural stop signal**, so the `<end>` state is unnecessary: generate open-ended and truncate at the first dst hit. This removes the diffusable stop-state entirely (no `bd_eos_deg` to calibrate). Retraining the uniform kernel with the plain $V \times V$ adjacency (no augmented `<end>` node), normal data, `refine_es`:

blk	no-EOS: hit / arr / valid / len err	EOS (§1.2): hit / arr / valid / len err
4	0.596 / 0.737 / 1.000 / 3.50	0.156 / 0.303 / 0.989 / 6.11
8	0.578 / 0.736 / 1.000 / 3.28	0.120 / 0.208 / 0.896 / 8.86
16	0.476 / 0.607 / 0.995 / 5.01	0.118 / 0.212 / 0.969 / 8.55
32	0.256 / 0.405 / 1.000 / 7.49	0.161 / 0.326 / 0.994 / 6.05

Dropping `<end>` roughly doubles-to-quadruples hit and arrival at every block size while keeping ~ 1.0 validity — a larger and more fundamental fix than the `eos_deg` sweep (§1.3), because there is no premature-stop mechanism at all. This is the recommended configuration for the uniform kernel in OD planning. (The EOS column's higher EM is an artifact: early-stopped fragments are trivially "shortest" for their own truncated endpoints, so EM flatters the broken model — hit and arrival are the honest columns. `<end>` would only be needed for *unconditional* generation, which has no destination to terminate on.) The advantage narrows at blk32, consistent with the general finding that large blocks hurt the uniform kernel.

1.5 Destination conditioning: prefix token vs embedding-only (mask kernel)

Does the destination need to be an in-context prefix token, or does conditioning it purely through the OD embedding (adaLN) suffice?

Retrained mask kernel, embedding-only (`-bd_no_dst_prefix`: dst removed from the sequence, origin stays as the path start), both at CFG 1.0, `refine_es`:

blk	prefix+emb: hit / valid / EM / len	embedding-only: hit / valid / EM / len
4	0.942 / 0.855 / 0.796 / 0.43	0.951 / 0.839 / 0.786 / 0.31
8	0.958 / 0.772 / 0.724 / 0.42	0.884 / 0.733 / 0.685 / 0.57

The dst prefix token is essentially unnecessary: at blk4 the two are a statistical tie (within per-run OD-sampling noise), at blk8 the prefix gives a small edge. This aligns with the DiT finding that adaLN conditioning (their winning mechanism) beats in-context conditioning — the embedding alone carries the destination to every block. So the design can be simplified: condition on dst purely as an adaLN signal, keeping the origin as the path start (cleaner canvas, no index-0 bookkeeping, no prefix block-slot overhead).

2. Single exceptional case with an "oracle" discriminator

Setup: (2.1) base diffusion trained on **normal** only; (2.2) discriminator trained on normal vs **except_0** only (10% of its data; val acc 86.6%, calibrated NLL 0.266) and used for **importance-weight guidance** (§4.2, reweight mode, N = 50); (2.3) tested on **except_0** — the same scenario the discriminator was trained on. Mask kernel, CFG w = 1.0, **raw** variant (no refinement — unaided model behavior).

blk	valid: unguided	valid $\lambda=2$	valid $\lambda=8$	hit: unguided	hit $\lambda=2$	hit $\lambda=8$
4	0.105	0.093	0.068	0.954	0.940	0.928
8	0.077	0.074	0.079	0.943	0.941	0.949
16	0.071	0.059	0.067	0.933	0.927	0.921
32	0.049	0.050	0.037	0.910	0.902	0.881

Finding (mask): even with a discriminator trained on the *exact test scenario* with 10× data, importance-weight guidance does not improve the mask kernel over unguided generation. Combined with §3 this is the decisive ablation: **the limitation is the guidance mechanism's interaction with over-confident proposals, not the discriminator's data or generalization.**

Uniform kernel with the oracle discriminator (reweight mode, $\lambda=2 / \lambda=8$; the 99-scenario-D and unguided columns repeat §3.2 for comparison).

Valid rate:

blk	oracle D	99-scenario D	unguided
4	0.539 / 0.572	0.461 / 0.514	0.233
8	0.544 / 0.596	0.510 / 0.528	0.216
16	0.548 / 0.604	0.493 / 0.546	0.239
32	0.367 / 0.459	0.296 / 0.335	0.085

Hit ratio (same runs):

blk	oracle D	99-scenario D	unguided
4	0.048 / 0.041	0.067 / 0.052	0.142
8	0.006 / 0.013	0.011 / 0.015	0.108
16	0.012 / 0.014	0.016 / 0.023	0.108
32	0.059 / 0.043	0.063 / 0.050	0.167

Finding (uniform): on the kernel where guidance responds, the oracle raises *validity* only modestly over the generalizing discriminator (+0.03–0.12) — 99-scenario D captures ≈85–90% of the oracle's achievable value, so scenario transfer is cheap when the mechanism works. But **both discriminators collapse the hit ratio** (0.142 → 0.05 at blk4; even the oracle does not escape this) — the validity gain and the arrival objective are traded, not jointly won (see §3.3).

3. Generalization to an unseen exceptional case

Setup: (3.1) discriminator trained on **99 exceptional cases** (**except_1..99**, **1% of each**) vs normal — val acc 81%, calibrated NLL 0.311; transfer check: given the unseen scenario's adjacency, it separates unseen-scenario paths from normal paths at **AUC 0.949**. (3.2) tested on **except_0** (never seen by the diffusion model or the discriminator; its adjacency is provided to the discriminator's GraphSAGE at inference). (3.3) importance-weight guidance (§4.2, reweight, N = 50), CFG w = 1.0, **raw** variant.

3.1 Mask kernel

Valid rate (unguided baseline right column):

blk	$\lambda=1$	$\lambda=2$	$\lambda=4$	$\lambda=8$	unguided
4	0.093	0.091	0.082	0.085	0.105
8	0.091	0.076	0.090	0.071	0.077
16	0.083	0.079	0.061	0.065	0.071
32	0.085	0.053	0.061	0.064	0.049

Hit ratio (same runs):

blk	$\lambda=1$	$\lambda=2$	$\lambda=4$	$\lambda=8$	unguided
4	0.936	0.936	0.942	0.955	0.954
8	0.956	0.949	0.938	0.944	0.943
16	0.930	0.911	0.910	0.932	0.933
32	0.899	0.888	0.914	0.906	0.910

Validity is flat (no λ response) at small blocks, with a small real gain only at blk 32 / gentle λ (0.085 vs 0.049); **hit ratio is unaffected by guidance throughout (0.89–0.96)** — on the mask kernel guidance neither helps nor hurts.

3.2 Uniform kernel (valid rate / hit ratio)

The `raw` variant has no refinement, so arrival = hit here; the second entry is the hit ratio:

blk	$\lambda=2$	$\lambda=8$	unguided
4	0.461 / 0.067	0.514 / 0.052	0.233 / 0.142
8	0.510 / 0.011	0.528 / 0.015	0.216 / 0.108
16	0.493 / 0.016	0.546 / 0.023	0.239 / 0.108
32	0.296 / 0.063	0.335 / 0.050	0.085 / 0.167

On the uniform kernel guidance raises validity (roughly doubles at blk 4–16, quadruples at blk 32, and responds to λ) **but drives the hit ratio down** (0.142 $\rightarrow$ 0.05 at blk4) — paths become graph-consistent but stop reaching the destination. The guidance target (except-consistency) and the task objective (arrival) are traded against each other; see §3.3.

3.3 Guidance-method comparison: reward-tilted vs kernel correction

The two §4.2 methods (plus granularity) on identical models / discriminator (99-scenario) / scenario (unseen except_0), valid rate on raw paths:

Valid rate:

config	unguided	reweight ($\lambda=2$)	ratio (per-block)	marginal (full-canvas)
mask blk4	0.105	0.091	0.093	0.096
mask blk32	0.049	0.053	0.067	0.053
grapheos blk4	0.233	0.461	0.253	0.232
grapheos blk16	0.239	0.493	0.237	0.232

Hit ratio (reaches dst unaided) — the cost side of the trade-off:

config	unguided	reweight ($\lambda=2$)	ratio (per-block)	marginal (full-canvas)
mask blk4	0.954	0.936	0.948	0.951
mask blk32	0.910	0.888	0.900	0.900
grapheos blk4	0.142	0.067	0.161	0.145
grapheos blk16	0.108	0.016	0.115	0.085

- `ratio` $\approx$ `marginal` everywhere (as expected — same estimator at different granularity), validating both implementations; both leave hit essentially at the unguided level.
- **The reward-tilted mode's uniform-kernel validity gain comes at a hit-ratio cost:** on grapheos it roughly halves hit (blk4 0.142 $\rightarrow$ 0.067; blk16 0.108 $\rightarrow$ 0.016) while doubling validity. It steers paths onto graph-consistent edges but away from reaching the destination — the guidance target (except-consistency) and the task objective (arrival) are being traded against each other, not jointly satisfied. On the mask kernel, where validity barely moves, hit is preserved across all modes.
- **The theoretically principled kernel correction barely moves the uniform kernel, while the "improper" reward-tilted mode doubles its validity.** The explanation is the discriminator's logit compression: label smoothing (needed for training stability, §4.2) caps the calibrated logit at $|\tilde{f}| \approx 3$, i.e. density ratios of at most $\approx e^3$ — but the *true* ratio $q_{\text{except}}/q_{\text{normal}}$ for a path through the removed region is orders of magnitude more extreme. The exponent-1 kernel correction inherits this compression; reweight's $\lambda > 1$ (and its per-step compounding) empirically re-expands it. In other words: **λ is compensating for a deliberately bounded ratio estimate** — a bias-variance trade made once at discriminator training time and paid at guidance time.
- Costs per path (mask blk4): marginal 3 ms, ratio 3 ms, reweight ≈ 36 ms (per-step D calls).

valid $\wedge$ hit — the metric that matters (a usable path both arrives AND is legal):

Reporting validity or hit alone is misleading; the joint rate $P(\text{valid} \wedge \text{hit})$ is the real success metric, and $P(\text{valid} | \text{hit})$ isolates route legality among arriving paths. On the unseen except_0:

config	metric	unguided	reweight	ratio	marginal
mask blk4	valid $\wedge$ hit	0.090	0.083	0.113	0.093
mask blk32	valid $\wedge$ hit	0.047	0.046	0.078	0.071
grapheos blk4	valid $\wedge$ hit	0.012	0.013	0.015	—
grapheos blk16	valid $\wedge$ hit	0.008	0.002	0.013	0.017
grapheos blk16	valid hit	0.082	0.111	0.325	0.347

Two findings the split tables hid: (1) **reward-tilted guidance does NOT improve usable paths on the uniform kernel** — its large validity gain (§3.2) came entirely from *suppressing arrivals*, so valid $\wedge$ hit stays flat or drops (grapheos blk16: 0.008 $\rightarrow$ 0.002). (2) **The kernel-correction methods (ratio/marginal) genuinely help**: they roughly double valid $\wedge$ hit on the uniform kernel and lift valid|hit $\sim 4\times$ (0.082 $\rightarrow$ 0.347) — arriving paths become far more legal. So the two guidance *families* diverge exactly on the metric that counts, even though they looked similar on validity alone. (The grapheos blk4 marginal cell was abandoned — that run hung for 4h+ and was killed; marginal mode on the graph kernel is prohibitively slow.)

3.4 Interpretation (mechanism, confirmed from both directions)

- Mask kernel: $\hat{x}_0$ predictions are extremely confident (train CE $\approx$ 0.03) $\rightarrow$ the N sampled candidates are near-clones $\rightarrow$ reweighting has nothing to choose between $\rightarrow$ flat at every λ / block size / discriminator (oracle included, §2).
- Uniform kernel: $\hat{x}_0$ is soft (CE $\approx$ 1.3–2.0) $\rightarrow$ candidates genuinely differ $\rightarrow$ the importance weights select graph-consistent ones $\rightarrow$ validity doubles and scales with λ .
- Conclusion: the importance-weight guidance is sound; its effectiveness is gated by the entropy of the proposal distribution.** For the mask kernel, the fix is proposal-side (e.g. adjacency-masked proposals using the scenario graph, targeted temperature, or brief proposal fine-tuning on the 1% exceptional data), with the reweighting doing realism selection on top.
- Baseline context (unguided OOD): validity collapses from 0.95 $\rightarrow$ 0.12 (blk 1) and 0.51 $\rightarrow$ 0.07 (blk 64) when moving normal $\rightarrow$ unseen exceptional; block size does **not** confer OOD robustness. Severity scales as expected: at remove-ratio 0.05 (100 edges), unguided valid is 0.234 vs 0.092 at ratio 0.1 (blk 4).

4. Formulations used in the code

4.1 Conditional generation

Canvas. A trajectory $p = (v_0=\text{ori}, \dots, v_{\{L-1\}}=\text{dst})$ is embedded in the canvas

```
x = [ dst, ori, v1, ..., v_{L-1}, <end>, <pad>, ... ]      (positions 0 and 1 are the condition)
```

padded to a multiple of the block size L' . The destination therefore conditions generation two ways: as an **in-context prefix token** (position 0, never noised when conditioned) and through a **conditioning embedding** $c = \text{MLP}_t(t) + \text{MLP}_{od}([e(\text{ori}); e(\text{dst})])$ injected via adaLN at every layer. With probability `drop_cond` = 0.1 a training sample is made fully unconditional (embedding $\rightarrow$ learned null vector AND prefix noised + in-loss), which enables `plan(None, None)`.

Block-diffusion factorization (BD3-LM): with blocks $x^1, \dots, x^B$ of size L' ,

$$p_\theta(x) = \prod_{b=1}^B p_\theta(x^b | x^{<b}),$$

each factor realized by discrete diffusion over block b conditioned on clean previous blocks. Training uses one forward pass over the concatenation $[x_t \parallel x_0]$ with the block-causal attention mask (noised block b attends to itself and to clean blocks $< b$), and an independent noise level $t_b \sim U(0,1]$ per block.

Mask kernel loss (absorbing / SUBS): each token of block b is masked independently with probability $t_b \sim U(0,1]$;

$$\mathcal{L}_{\text{mask}} = \mathbb{E}_{t, x_t} \left[\sum_{\ell: x_{t,\ell} = [\text{MASK}]} -\frac{1}{t_{b(\ell)}} \log p_\theta(x_{0,\ell} | x_t, x^{<b}, c) \right]$$

(positions: loss over the whole canvas from position 2 on, including `<end>` / `<pad>`; the prefix is included only for unconditional samples).

Uniform (graph CTMC) kernel loss (D3PM ELBO): forward kernel $Q_t = \exp(G \cdot \beta_t)$ with $G = A' - D'$ over the EOS-augmented state space (A' adds a virtual `<end>` node connected to all vertices with weight $2/|V|$; symmetric $\Rightarrow$ uniform limiting distribution). With cumulative kernels $\bar{Q}_t$, the true and predicted posteriors are

$$q(x_{t-1} | x_t, x_0) \propto (Q_t^\top \mathbb{1}_{x_t}) \odot (\bar{Q}_{t-1} \mathbb{1}_{x_0}), \quad p_\theta(x_{t-1} | x_t) \propto (Q_t^\top \mathbb{1}_{x_t}) \odot (\bar{Q}_{t-1} \hat{x}_0)$$

$$\hat{x}_0 = \text{softmax } f_\theta(x_t, t, x^{, c), \quad \mathcal{L}_{\text{graph}} = \text{KL}(p_\theta \| q) + \lambda_{\text{ce}} \text{CE}(\hat{x}_0, x_0) + \lambda_{\text{con}} \text{Conn}(\hat{x}_0; A)$$

with per-block integer $t_b \in \{1..T\}$, $T = 100$.

EOS-augmented state space — the mathematics of `bd_eos_deg`. To let the uniform-state kernel terminate, the state space is extended with one virtual `<end>` state. With $d := \text{bd_eos_deg}$ and per-vertex weight $w = d/|V|$, the augmented adjacency and CTMC generator are

$$A' = \begin{pmatrix} A & w \mathbb{1} \\ w \mathbb{1}^\top & 0 \end{pmatrix}, \quad L' = A' - \text{diag}(A' \mathbb{1}), \quad Q_t = \exp(L' \beta_t), \quad w = \frac{d}{|V|}$$

so d is the **total degree of the `<end>` node**, split evenly across all vertices. Its meaning:

- **Rate interpretation.** In the forward CTMC, a real vertex leaks into `<end>` at instantaneous rate $w \cdot \beta_t$, and `<end>` leaks back into each vertex at the same rate (total out-rate $d \cdot \beta_t$). Setting $d \approx$ average vertex degree $\bar{d} = 2|E|/|V| \approx 2.88$ makes `<end>` mix like an ordinary vertex (the $d = 2.0$ default); $d \ll \bar{d}$ makes it nearly isolated, $d \gg \bar{d}$ makes it an attractor.
- **Stationarity is preserved.** A' is symmetric $\Rightarrow L' \mathbb{1} = 0$ and $L' = L'^\top$, so the uniform distribution over the $|V|+1$ states remains the limiting distribution — the kernel is still a *uniform-state* diffusion; only the mixing pattern changes.
- **Effect on sampling.** The forward marginal into EOS grows with d ($q(x_t = \text{end} | x_0 = v) = [\exp(L' \cdot B_t)]_{\text{end}, v} \approx w \cdot B_t$ for small cumulative noise $B_t = \sum \beta_s$), and the reverse posterior $q(x_{t-1} | x_t, \hat{x}_0) \propto (Q_t^\top \mathbb{1}_{x_t}) \odot (\bar{Q}_{t-1} \hat{x}_0)$ inherits this reachability. **d therefore directly scales the prior propensity to propose `<end>` during generation** — the observed premature stopping at $d = 2.0$ (§1.2) is this term dominating, which motivates the calibration sweep over d (§1.3).
- **Bookkeeping.** The `<end>` state index equals the `<end>` token id ($= |V|$), so one vocabulary serves both kernels. In the final adjacency-constrained decode the binarized A' makes `<end>` a legal successor of every vertex regardless of d — d shapes the diffusion dynamics, not decode legality.

Classifier-free guidance (evaluated, then disabled): with conditional logits z_c and null-condition logits z_u ,

$$z_{\text{CFG}} = z_u + w(z_c - z_u).$$

Empirically $w = 1.0$ (pure conditional) is best — see §1.1.

Sampling (semi-AR): generate block b given committed blocks $< b$; mask kernel uses the first-hitting sampler (reveal one masked token per step, $t \leftarrow t \cup \{1/\#\text{masked}\}$); graph kernel runs the full T -step reverse with a 10-sample Monte-Carlo posterior and an adjacency-constrained left-to-right decode of the finished block. Stop when `dst` (hit) or `<end>` (miss) is emitted; misses may be refined (§1 variants).

4.2 Importance-weight guidance (discriminator)

Discriminator. $f_\phi(x_{\text{path}}, A) \rightarrow \mathbb{R}$ scores a (partial) path against a *given* adjacency A (GraphSAGE-conditioned node embeddings $\rightarrow$ conv encoder $\rightarrow$ pooled logit). Trained with BCE + label smoothing $\epsilon = 0.05$ (bounds the optimal logit: $|f| \leq \log((1-\epsilon)/\epsilon) \approx 2.94$, which fixes the logit-blowup instability), weight decay $1e-4$, model selection on validation BCE, and **post-hoc temperature calibration** T (fitted on held-out data); the calibrated score is $\tilde{f} = f/T$. Label 1 = exceptional, 0 = normal; **both classes are scored under the exceptional scenario's adjacency**, so the learned feature is graph-relative ("does this path fit the given graph") and transfers to unseen scenarios whose adjacency is supplied at test time. Training paths are randomly truncated at block boundaries so $\tilde{f}$ acts as a prefix value function.

Two guidance methods are implemented and selectable at test time (`-disc_mode`), sharing the same discriminator. They differ in their **target distribution**: (a) is a *reward-tilted* version of the model's own distribution; (b) is a *kernel correction* whose target is the exceptional data distribution q itself.

(a) Reward-tilted guidance (`-disc_mode reweight` — used in §2 and §3)

At every denoising step of the current block b , the model's $\hat{x}_0$ prediction is replaced by a discriminator-tilted mixture.

Step 1 — candidates. Draw N candidate completions of the block from the model's per-position $\hat{x}_0$ distribution ($N = 50$):

$$\hat{x}_0^{(i)} \sim \prod_{\ell \in b} p_\theta(x_{0,\ell} | x_t, x^{, c), \quad i = 1, \dots, N$$

Step 2 — importance weights. Score each candidate, prepended with the committed prefix, using the calibrated discriminator $\tilde{f}_\phi = f_\phi / T_{\text{cal}}$ under the target scenario's adjacency A :

$$w_i = \frac{\exp(\lambda \tilde{f}_\phi(x^{ \| \hat{x}_0^{(i)}; A))}{\sum_{j=1}^N \exp(\lambda \tilde{f}_\phi(x^{ \| \hat{x}_0^{(j)}; A))}$$

Step 3 — guided mixture. Form the weighted empirical distribution per position ℓ and state v :

$$\tilde{p}(x_{0,\ell} = v) = \sum_{i=1}^N w_i \mathbb{1}[\hat{x}_{0,\ell}^{(i)} = v]$$

Step 4 — reverse update. Use $\tilde{p}$ in place of p_θ in the reverse update (reveal probabilities for the mask kernel; the CTMC posterior for the graph kernel). A single application approximates the **tilted distribution**

$$\tilde{p}_\lambda(x_0^b | x_t, x^{<b}) \propto p_\theta(x_0^b | x_t, x^{<b}) \cdot \exp(\lambda \tilde{f}_\phi(x^{<b} \| x_0^b; A)).$$

Committed positions inside the block are held fixed across candidates. **Target caveat:** because the same fixed tilt $\exp(\lambda \tilde{f})$ is re-applied at every denoising step, the tilts compound across steps — the overall sampler is a value-guided twisted process without a closed-form target; λ is a heuristic strength knob. (Applied at block scope, at all t ; the repo's original `sample_with_len_disc_mn` applied it only for $t < 10$ on the full canvas.)

(b) Importance-weight kernel correction (`-disc_mode ratio` — the paper's method)

Here the weight is not a reward but the **density ratio** that turns the model's proposal kernel into the *target distribution's* reverse kernel. Writing q for the exceptional data distribution and using the same forward corruption for both:

$$q(x_{t-1} | x_t) = \mathbb{E}_{x_0 \sim p_\theta(x_0 | x_t)} \left[\frac{q(x_0 | x_t)}{p_\theta(x_0 | x_t)} q(x_{t-1} | x_t, x_0) \right], \quad \frac{q(x_0 | x_t)}{p_\theta(x_0 | x_t)} \propto \frac{q(x_0)}{p_\theta(x_0)} = \frac{D_\phi(x_0)}{1 - D_\phi(x_0)}.$$

Because the ratio measures exactly the mismatch remaining at each step given x_t , applying the correction at every step is consistent (no compounding): composing corrected kernels from $t = T$ to 0 yields samples from **q itself** — a fixed target with **no λ** (exponent 1).

Implementation (the marginal-sample estimator). Per block: (1) generate N candidate completions **once** with the model's own unguided block sampler; (2) score them **once** with the calibrated discriminator, giving log-ratios $\log r_n = \tilde{f}_\phi(x^{<b} \| x_0^{(n)}; A)$; (3) re-generate the block from the candidate mixture with per-step **closed-form forward-kernel weights** (no further model or discriminator calls):

$$w_n(t) = \frac{q(x_t | x_0^{(n)}) r_n}{\sum_{m=1}^N q(x_t | x_0^{(m)}) r_m}, \quad \bar{x}_{0,\ell} = \left[\sum_n w_n(t) \mathbb{1}\{x_{0,\ell}^{(n)} = k\} \right]_k,$$

with $q(x_t | x_0^{(n)})$ available in closed form: for the graph kernel $q(x_t | x_0) = \prod_\ell [Q_t]_{\{x_0, \ell \rightarrow x_t, \ell\}}$; for the absorbing kernel the shared t -factors cancel and $w_n(t)$ reduces to r_n restricted to candidates **consistent with all already-revealed positions**. Cost: N block-generations + one discriminator batch per block (vs N discriminator calls per step for mode (a)).

Block-scope justification. Scoring prefix $\parallel$ candidate and self-normalizing across candidates that share the prefix yields exactly the *conditional* ratio $q(x^{<b} | x^{<b}) / p_\theta(x^{<b} | x^{<b})$ — the prefix factor of the joint ratio is constant across candidates and cancels.

Full-canvas variant (`-disc_mode marginal`). The same kernel correction instantiated exactly as the marginal-sample estimator (Eqs. 31–36 of the companion derivation): N **complete candidate paths** are generated once by the model's own semi-AR sampler — the candidate generator's block size is a free choice, and small blocks give the best pools — scored once by the discriminator, and the output is sampled by a **network-free** reverse process over the whole canvas (one joint noise level t) with weights $w_n(t) \propto q(x_t | x_0^{(n)}) \cdot \exp(\tilde{f}_n)$. Cost per query: N candidate generations + one discriminator batch; the guided reverse itself makes no model calls. The per-block variant (b) refreshes candidates each block (conditioned on the committed prefix), which is what keeps small-block *following* consistent; the full-canvas variant avoids the issue by having no committed prefix beyond `[dst, ori]`.

Shared-discriminator approximation. All modes use the same normal-vs-exceptional discriminator; the kernel-correction theory calls for q/p_θ (negatives = model generations). The substitution is justified by the optimality assumption $p_\theta = q_{\text{normal}}$ (the model is trained on normal data); the residual gap is the model's own artifacts (e.g. invalid edges present in p_θ but not in normal data), which this discriminator neither rewards nor penalizes.

(A third variant — SMC-style select-one-of- n block resampling — was evaluated early on, gave the mask kernel flat results like mode (a), and has been removed from the code.)

5. Hyperparameter glossary

5.1 Diffusion model / training

Name (flag)	Symbol	Value used	Meaning
block_size	L'	1–64 (swept)	Tokens per block. Generation is autoregressive across blocks and diffusion within a block. $L'=1 \approx$ autoregressive model; $L' \geq \text{canvas} \approx$ full-canvas diffusion (MDLM).
kernel	—	mask / graph	Within-block corruption process: absorbing [MASK] replacement vs graph CTMC transitions $\exp((A-D)\beta_t)$.
max_T	T	100	Number of discrete reverse steps for the graph kernel (the mask kernel uses continuous $t \in (0,1)$).
beta_lb, beta_ub	β	$1e-4 \dots 10$	CTMC rate schedule endpoints; β_t interpolates linearly over T steps (graph kernel only).
bd_eos_deg	—	2.0	Total edge weight of the virtual <end> node added to A for the graph kernel (per-vertex weight = $2/ V $). Controls how fast the EOS state mixes in the forward process — currently too permissive (early stopping, §1.2).
drop_cond	p_uncond	0.1	Probability that a training sample is made fully unconditional: OD embedding $\rightarrow$ null vector AND the [dst, ori] prefix is noised and included in the loss. Enables unconditional generation and CFG.
guidance_scale	w	1.0 (best)	Classifier-free guidance weight: logits = $u + w \cdot (c - u)$. $w=1$ uses the pure conditional model; $w>1$ extrapolates (harmful here, §1.1).
lam_ce, lam_con	$\lambda_{ce}, \lambda_{con}$	1.0, 1.0	Weights of the auxiliary cross-entropy and graph-connectivity losses in the graph-kernel objective.
lr, bs, n_epoch	—	$5e-4, 32, 1$	Adam learning rate, batch size, epochs (~48k iterations over 1.54M training paths).
bd_hidden_dim / n_layers / n_heads / cond_dim	—	$256 / 6 / 8 / 128$	Transformer denoiser size ($\approx 6.8M$ parameters).
od_max_len	—	100	Cap on generated path length; canvas budget = $\text{ceil}((\text{od_max_len}+2)/L')$ blocks.

5.2 Sampling / evaluation

Name (flag)	Symbol	Value used	Meaning
length_mode	—	open	open : generate blocks until dst/ <end> appears (no length information). oracle : ground-truth length only sets the block budget.
eval_num, batch_traj_num	—	1000, 200	Number of evaluated OD pairs and planning batch size.
refine variant	—	raw / nx / es	Post-processing applied before scoring: none / networkx shortest-path patch to dst (guarantees arrival) / eval_shortest.refine() (dst-adjacent early-cut + loop removal; §1).
num_mc_samples	—	10	Monte-Carlo samples of $\hat{x}_0$ used to estimate the graph kernel's reverse posterior at each step.

5.3 Discriminator / guidance

Name (flag)	Symbol	Value used	Meaning
<code>disc_except_frac</code>	—	0.01 (§3), 0.10 (§2)	Fraction of each exceptional scenario's paths used for discriminator training (limited-data setting).
<code>disc_label_smooth</code>	ϵ	0.05	Label smoothing in the BCE targets $\{\epsilon, 1-\epsilon\}$. Gives the loss a finite minimizer, bounding the optimal logit at $\pm \log((1-\epsilon)/\epsilon) \approx 2.94$ — the fix for the logit-blowup instability.
<code>disc_wd</code>	—	1e-4	Discriminator weight decay (second stabilizer against logit growth).
calibration temperature	T_{cal}	≈ 0.71 (fitted)	Post-hoc scalar fitted on held-out data; the calibrated score $\tilde{f} = f/T_{\text{cal}}$ makes $\exp(\tilde{f})$ approximate a true density ratio (Guo et al. 2017).
<code>disc_scale</code>	λ	1–8 (swept)	Guidance strength: importance weights $w_i \propto \exp(\lambda \cdot \tilde{f})$. $\lambda=0$ disables guidance; larger λ sharpens selection toward exceptional-consistent candidates.
<code>n_importance_samples</code>	N	50	Number of candidate block completions drawn from $\hat{x}_0$ per denoising step for the importance-weighted mixture (§4.2). Larger N = better coverage of the proposal distribution, linearly more discriminator forwards.
<code>disc_mode</code>	—	reweight	Test-time guidance method. <code>reweight</code> : reward-tilted per-step $\hat{x}_0$ mixture with $\exp(\lambda \cdot \tilde{f})$ weights (§4.2a; used in §2/§3). <code>ratio</code> : kernel correction — candidates refreshed per block, per-step forward-kernel $\times D/(1-D)$ weights, exponent 1, targets q itself (§4.2b). <code>marginal</code> : full-canvas kernel correction — N complete candidate paths generated once, one discriminator pass, network-free guided reverse (§4.2b).

Artifacts

- Checkpoints: `sets_model/BD2_porto_v3-0.1_{mask,grapheos}_blk{1..64}_*.pth`, `BDDISC_v4-0.1_ex1-99_frac0.01`, `BDDISC_v4-0.1_ex0only_frac0.10`, `BDDISC_v4-0.05_ex1-99_frac0.01`
- Results: `sets_res/cfg1_all/` (§1), `sets_res/abl_ex0/` (§2), `sets_res/rw3_ex0/` (§3), `sets_res/ex0_cfg1/` (unguided OOD baselines), `sets_res/gen05_ex0/` (0.05 severity), each with per-path EM/PC records under `em_pc/`
- Code: `main_bd.py`, `models_seq/bd_models.py`, `models_seq/bd_disc.py`, `models_seq/bd_trainer.py` (commit 707215b)